Volopay e-invoicing prototype (UAE)

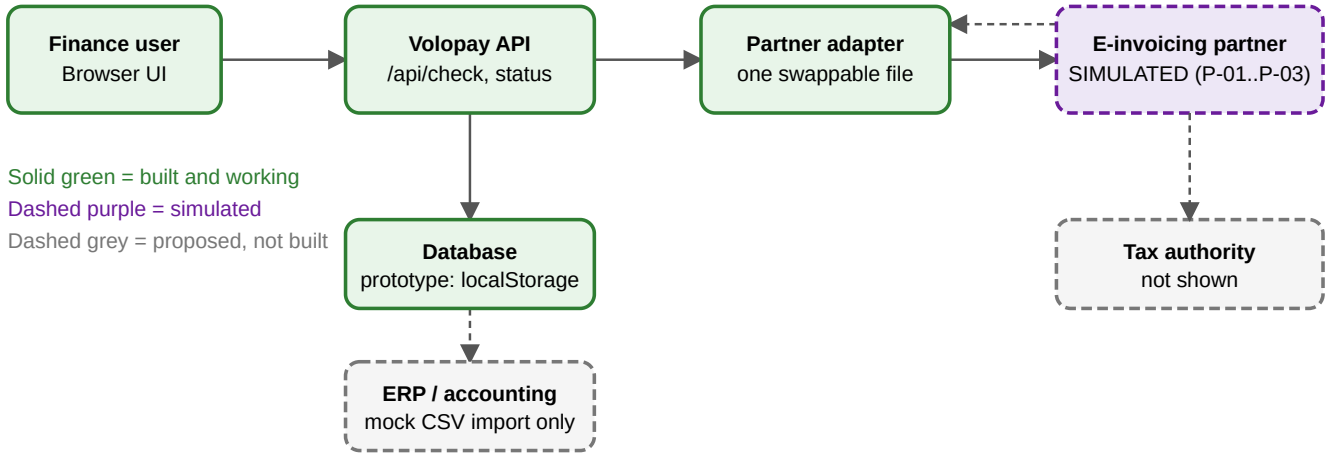
Challenge 3 of the Founder's Office assignment. Everything that touches a provider, tax authority or accounting tool is **simulated**.

1. Live prototype

Open the prototype: https://prototype-volopay.vercel.app/

Source code: github.com/Jha-Komal/prototype-volopay. It opens in a browser with no download or setup. Use **Reset sample data** on the Invoices page to start again. Demo path: INV-1001 (happy path), INV-1002 (blocked by V-01, fix the TRN in the editor), INV-1003 (blocked by V-02 and V-03, one-click fixes), INV-0999 (rejected, DUPLICATE_INVOICE), INV-1005 (rejected, BUYER_NOT_FOUND). After an invoice is accepted, record a payment on its page to see matching.

2. Architecture



How data moves. The user fills an invoice in the browser, which stores it (browser storage in the prototype, Postgres in production). **Run checks** sends it to `/api/check`, the authoritative Volopay checks (V-01 to V-05). Only invoices that pass can be submitted. Submit calls the partner adapter, which calls the simulated provider endpoint. The provider returns accepted or rejected with a code. The API stores the response as a status event and the invoice page shows it in the timeline. **Where validation happens:** in the browser while typing (hints), on the Volopay API before sending (authoritative), and at the provider after sending (simulated rules).

Part	Built in this prototype	Proposed for production
UI, checks, status timeline, payment matching, incoming match, CSV import	Built and working	Same, hardened, with real auth
Database	Browser localStorage	Postgres with audit history

E-invoicing provider	Simulated endpoint	Accredited provider via its API, behind the same adapter interface
ERP / accounting link	Mock CSV import	Real connectors
Tax authority	Not shown	Reached only through the provider

3. Build vs partner decisions

Volopay builds: invoice creation, pre-send validation, status tracking and error handling, the receive-side matching, collections matching, and the accounting connectors. These sit close to the finance user and are where Volopay differentiates. **The partner provides:** the legal link to the tax system. In the UAE only an accredited provider may submit, so a partner is required. The prototype hides the provider behind one interface (`PartnerAdapter`) so a real accredited provider can replace the simulator by swapping one file. This follows the Challenge 1 strategy of adding e-invoicing to the AR and AP products. The named provider candidates come from the Challenge 1 research and should be inserted here. The prototype does not name any provider as connected.

4. What works vs what is simulated

Works	Simulated or not built
Create, edit and save invoices. Live hints. Server-side checks V-01 to V-05 with rule IDs and the field to fix. Blocked invoices cannot be submitted. Status timeline. Fix and resubmit. Payment recording and matching (short, over, exact). Incoming invoice matching and approval. CSV import to drafts. Reset sample data.	The provider, its responses, reference IDs and the 2 to 3 second delay. The tax authority. The accounting tool (CSV only). The payment (no money moves). Storage is per browser. No login. Rules marked assumed are invented for the demo.

5. Tools, assumptions, limitations, next steps

Tools: Next.js 16 (App Router), React 19, TypeScript, Tailwind 4, browser localStorage. Built with Claude Code, deployed on Vercel.

Assumptions: a TRN has 15 digits (secondary sources, not confirmed against the FTA); standard VAT is 5%; each line needs a VAT category; invoice numbers look like INV-0000 (invented). The Rules page marks every rule as sourced, assumed or simulated. The official KPMG/FTA field guidance could not be opened from the build environment, so the required-fields list covers a subset of the 51 mandatory fields and is not confirmed.

Limitations: single browser, no authentication, no multi-seller uniqueness, no PDF or XML output, no credit notes, no retry for temporary provider errors, English only, desktop-first.

Next steps to launch: (1) confirm rules against the FTA and Ministry of Finance sources; (2) select and contract an accredited provider and map the invoice model to its schema; (3) replace the simulator with the real adapter and handle async callbacks and retries; (4) move storage to a production database with audit trail; (5) add authentication and multi-tenant access; (6) build the accounting connectors; (7) security review and sandbox certification with the provider.

6. Where AI was used and how it was checked

Claude Code wrote the application code from the spec in the repository README, and drafted this document. Checks: the lint, type check and production build pass; every seed scenario (accept, blocked by checks, rejected then fixed and accepted), the payment match, incoming approval and CSV import were run end to end in a real browser; and rules from AI or secondary sources are labelled as assumed rather than stated as official. The author should re-verify the rules against official sources before relying on them.